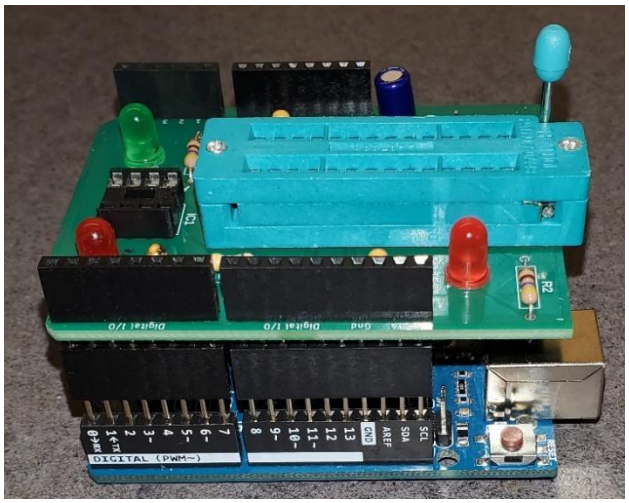
Bare Bones Programming Shield Assembly and Theory of Operation Manual

By: Jim Schrempp and Bob Glicksman; v1.1, 3/13/2026

NOTICE: Use of this document is subject to the terms of use described in the document "Terms_of_Use_License_and_Disclaimer" that is included in this release package. This document can be found at:

https://github.com/TeamPracticalProjects/Bare_Bones_Programming_Shield/blob/main/Terms_of_Use_License_and_Disclaimer.pdf



© 2026 Team Practical Projects, Bob Glicksman, and Jim Schrempp. All rights reserved.

TABLE OF CONTENTS.

TABLE OF CONTENTS. 1

OVERVIEW. 2

WHY “BARE BONES”? 2

WHAT IS IN THIS PROJECT? 3

WHAT THE PROGRAMMER IS MADE OF. 3

THE SHIELD BOARD. 4

BUILDING THE SHIELD BOARD. 6

CONFIGURING THE ARDUINO IDE FOR PROGRAMMING “BARE BONES” CHIPS. 9

Adding support for the ATmega328(P) “Bare Bones” chip. 9

Adding support for the ATtiny85 “Bare Bones” chip. 11

Reset the Arduino IDE. 11

FLASHING THE ARDUINOISP PROGRAM TO THE UNO. 13

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to “Terms of Use, License and Disclosure”*

OVERVIEW.

This document contains the Theory of Operation and Assembly Instructions for the “Bare Bones” Programmer (BBP) from Team Practical Projects.

The BBP leverages built-in Arduino tools that enable the user to develop code for “bare bones” AVR microcontroller chips using the Arduino IDE and a custom shield board for an Arduino Uno R3[1]. The user should carefully read through this document and follow all instructions herein in order to successfully create a programmer for ATtiny85 and Atmega328(P) “bare bones” chips.

WHY “BARE BONES”?

In the context of this document, “bare bones” means using an ATtiny85 (or 45 or 25) or an Atmega328(P) chip for an embedded project without any external components to make the microcontroller chip run. It will not be necessary to use a board-level (Arduino Uno) or module level (Arduino Nano) product if a “bare bones” microcontroller chip is used. The advantages of using a “bare bones” chip vs a board or module level component are:

- Lower cost.
- Smaller footprint.
- Wider range of power supply voltages (e.g. when powering from batteries)
- Ability to power down the microcontroller to extremely low power draw while retaining the program and data state.

This project supports the following “bare bones” microcontroller chips:

- ATtiny85. This is a small (8 pin) AVR microcontroller that is preferred when board space is at a premium. ATtiny25 and ATtiny45 versions are also supported, however these have smaller program and data memories and the small difference in cost may not be worth the limitation on memory.
- ATMega328(P). This is the chip that is used on the Arduino Uno (R3 and before) and also on the Nano. Both the -P and non-P version of the chip are supported; however, the default file that provides the Arduino IDE with support for this chip is for the -P version. The -P version of the chip contains a few more machine language instructions and powers down to a lower level than does the non-P version.

Note: Other AVR chips may be compatible with this project, however, instructions for other AVR chips are not provided nor have other AVR chips been tested with this project.

[1] This project has been fully tested using an Arduino Uno R3. It will likely run fine on an older Uno board or a clone, as long as the Uno board uses an ATmega328P chip. It will likely *NOT* work on an Uno R4, which uses a 32 bit 3.3 volt microcontroller.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to “Terms of Use, License and Disclosure”*

WHAT IS IN THIS PROJECT?

The standard Arduino IDE contains an example program called “ArduinoISP”. When loaded onto an Arduino Uno, this program turns the Uno into a programmer that can upload code to a “bare bones” AVR chip using in-circuit serial programming (ISCP). ISCP uses the !RESET and SPI pins on the “bare bones” microcontroller chip to write programming data into the microcontroller’s program memory. No bootloader is required[2]. In order to use the Arduino IDE to develop and compile code for the “bare bones” microcontroller chip, support for the chip must be added to the IDE. Instructions for doing this are contained in this document.

This project contains a custom-made “shield” board for an Arduino Uno. The shield board mates with an Uno to create a complete programmer for DIP versions of an:

- 8 pin ATtiny 25/45/85.
- 28 pin ATmega 328 or 328P

The custom shield board provides sockets that mount these chip-level products for programming via the Arduino IDE and the attached Uno. No manual wiring is necessary. The shield board also has an LED connected to each of the programming sockets. The LED is connected to the correct pin of each type of supported microcontroller so that the LED can be controlled via Arduino digital I/O number 4. This allows the user to test a new (factory fresh) chip using the standard Arduino “blink” program while the chip is still in the programmer. The actual application program can then be compiled and flashed onto the chip.

WHAT THE PROGRAMMER IS MADE OF.

The picture below is that of an assembled Programmer.

[2] The process described in the User Manual requires flashing a bootloader to the “bare bones” chip in order to properly set the fuses. The bootloader itself is not used.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to “Terms of Use, License and Disclosure”*

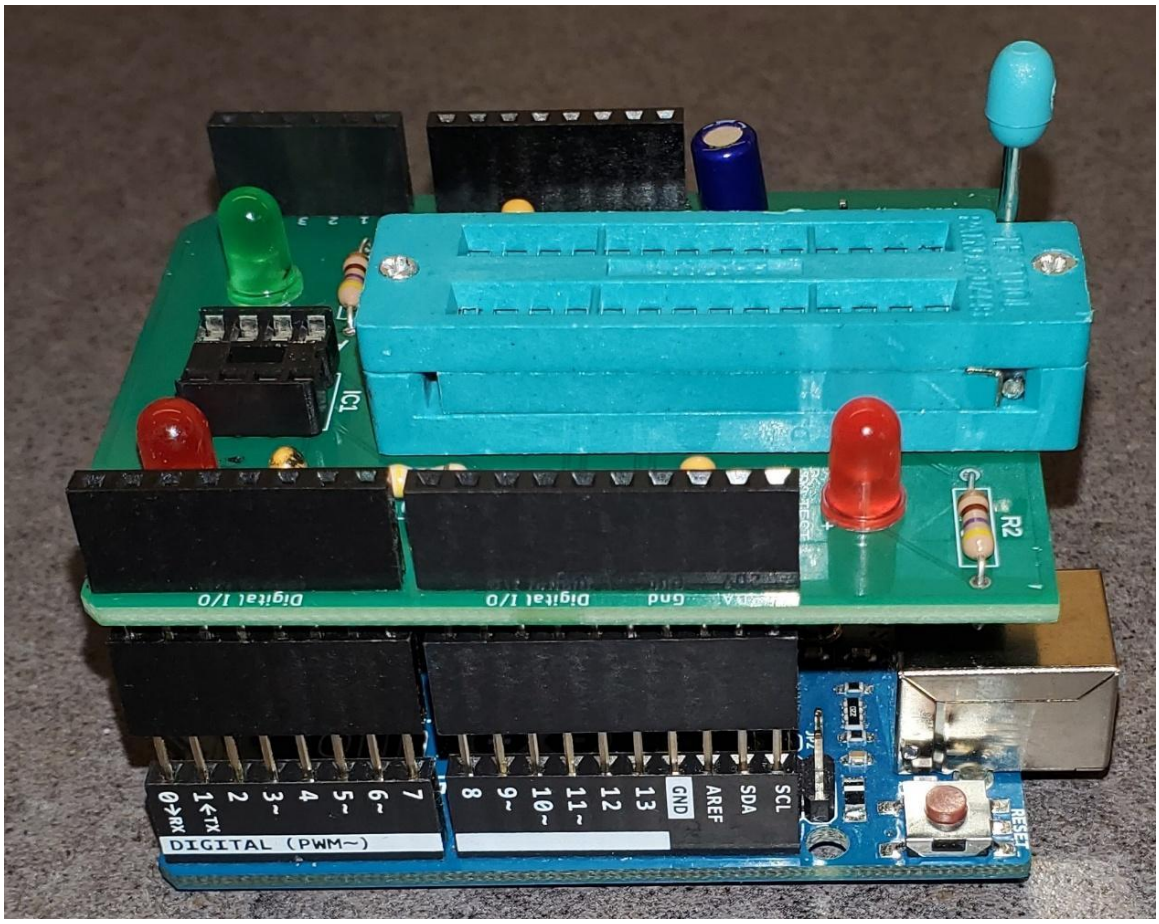


Figure 1. Assembled Programmer.

The programmer consists of an Arduino Uno (R3), a set of stacking headers, and an assembled custom shield board. The picture in figure 1 shows an optional ZIF socket for programming 28 pin Atmega328(P) chips. An ordinary 28 pin socket may be used; however, care must be taken when inserting the large chip into the socket in order that the pins not get bent in the insertion process.

The entire programmer is powered using the Uno's USB socket. Normally, an A-B USB cable connects this socket to a USB port on the computer that runs the Arduino IDE and is used to develop and compile/upload the resulting program.

THE SHIELD BOARD.

Figure 2 contains a schematic of the shield board.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

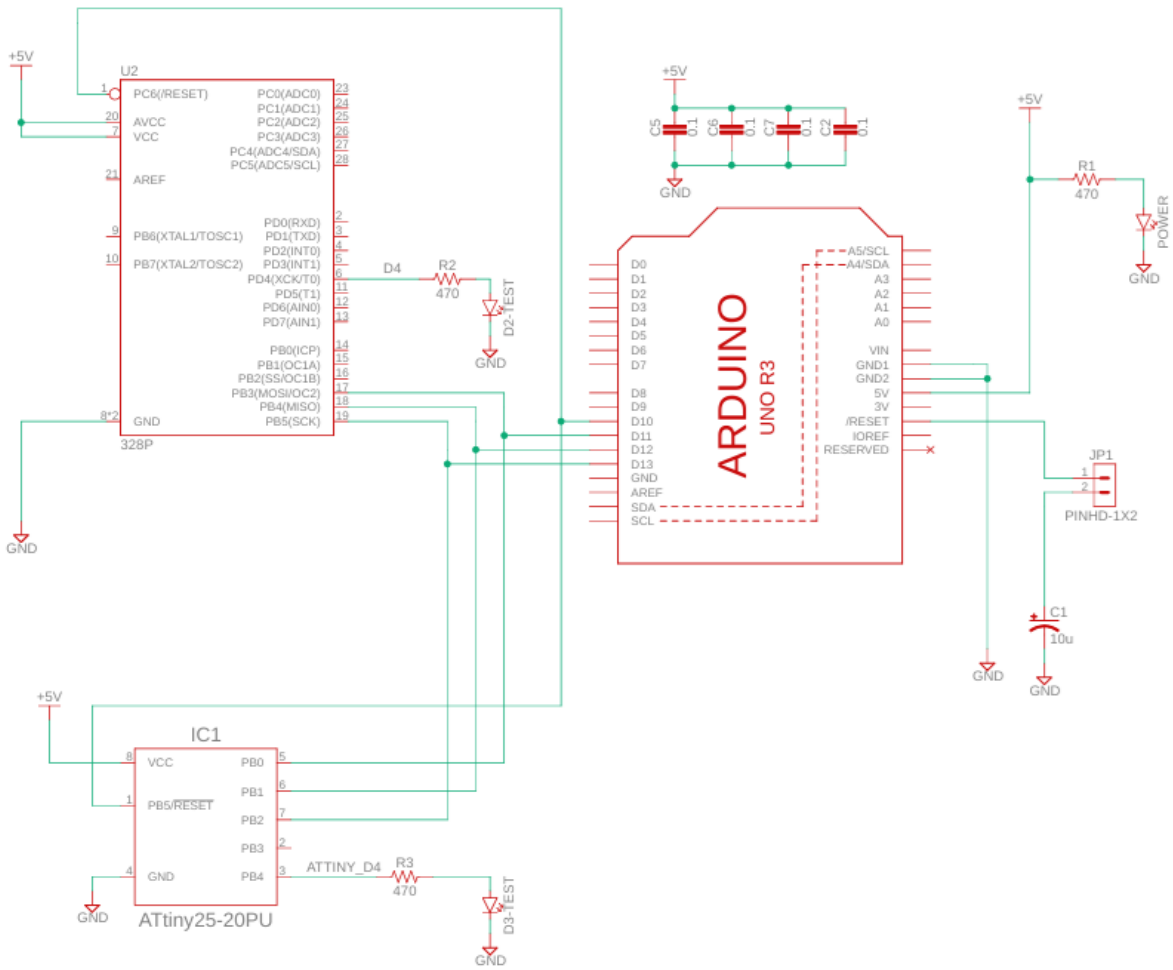


Figure 2. Shield Board Schematic.

The shield board contains a set of standard Arduino stacking headers. The following Arduino Uno signals are used on the shield board:

- Ground (GND).
- 5 volt power.
- !RESET
- D10
- D11
- D12
- D13

ICSP programming involves the !RESET pin on the microcontroller as well as the SPI pins: MISO, MOSI, SCLK. The ArduinoISP program that is run on the Uno uses the following Uno pins to connect to the relevant ATmega and ATtiny pins:

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

Arduino UNO Pin	Atmega328(P) Pin	ATtiny25/45/85 Pin
D10	1 (!RESET)	1 (!RESET)
D11	17 (MOSI)	5 (MOSI)
D12	18 (MISO)	6 (MISO)
D13	19 (SCLK)	7 (SCLK)

In addition, the shield board contains the following components:

- Decoupling capacitors (C2, C5, C6, C7).
- A green power indicator LED and its associated current limiting resistor.
- Two red LEDs and their associated current limiting resistors. The LED circuit on the 28 pin socket is connected to chip pin 6 while the LED circuit on the 8 pin socket is connected to chip pin 3. Both of these chip pins map to Arduino Digital Pin 4 in the Arduino IDE programming environment.
- A 2-pin header and a 10 uF capacitor. NOTE: a jumper should NOT be inserted into the 2-pin header. *Older instructions for programming bare-bones chips required a 10 uF capacitor be inserted between the Uno's !RESET pin and GND. This capacitor prevented the Uno from being reset when a new program was uploaded. The modern Arduino IDE does not reset the Uno when used according to the instructions in the User Manual that accompanies this project.*

BUILDING THE SHIELD BOARD.

The following parts are needed for the shield board:

- **Blank printed circuit board (PCB)**. See: https://github.com/TeamPracticalProjects/Bare_Bones_Programming_Shield/tree/main/Hardware for Fusion 360 electronic CAD files (.fsch and .fbrd) and a .zip archive containing manufacturing files. The manufacturing files are suitable for uploading to jlcpcb.com, and should also work for pcbway.com, oshpark.com, and other manufacturers.

- 2 each **Arduino standard stacking header kit**. https://www.adafruit.com/product/85. Also available from Digikey.com.

- **2 pin male pin header** (optional).

- **10 uF electrolytic capacitor** (10 volt or higher)(optional).

- **3 mm Green LED**

- 2 each **3 mm Red LED**

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

- 3 each **470 ohm resistor** (1/8 watt or ¼ watt axial)
- 4 each **0.1 uf capacitor** (10 volt or higher, 0.1" lead spacing)
- **28 pin (2 x 14) DIP socket** (or ZIF socket)
- **8 pin (2 x 4) DIP socket**

NOTE: *The 2 pin male pin header accepts a 0.1" jumper. When the jumper is inserted, it connects the 10 uf capacitor across the Uno's !RESET pin and GND. This component prevents the Uno from resetting when a program is uploaded to the Uno. When the Uno is running the ArduinoISP program, it should not be reset when the user's code is sent to the Uno, as resetting the Uno causes the Uno's bootloader program to run and uploads the code to the Uno, rather than to the chip being programmed. The Arduino IDE now supports an "upload using programmer" option which does not reset the Uno when uploading a user program to the bare bones chip. Therefore, the capacitor is no longer needed and the jumper that connects it into the circuit should be left open.*

Assembling the components on the printed circuit board is relatively straightforward. Only simple through hole components are used and the components can easily be hand soldered to the board. The only tricky part is soldering the stacking headers to the board, as the female tops of the headers must be flush with the board so that the male stacking pins are at true right angles to the board. Figure 3 shows a recommended method for aligning the header pins for soldering.

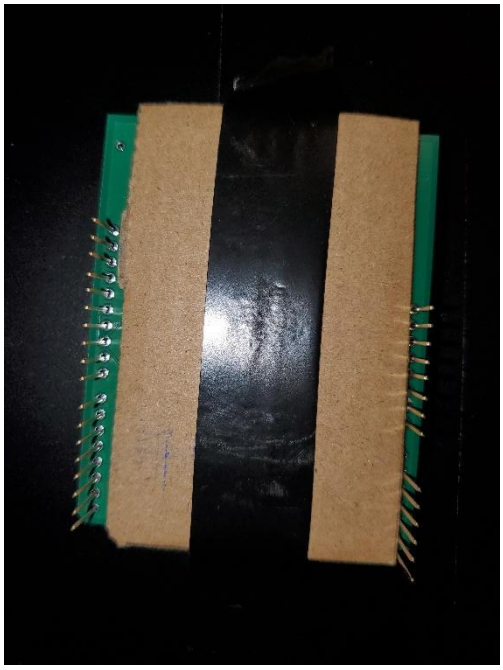


Figure 3. Aligning the Headers for soldering.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

Referring to figure 3:

- Cut out a piece of cardboard to protect the printed circuit board solder pads.
- Insert the stacking headers onto the component (top) of the board with the male pins sticking through.
- Carefully invert the board and place the top of the board on a flat surface. Make sure that none of the headers has fallen off of the board.
- Place the cardboard cutout over the solder pads on the bottom of the board and tape the board firmly down to the flat surface, making sure that the stacking header male pins are straight up (at right angles to the board).
- CAREFULLY solder the outermost pins on each stacking header, making sure that the pins remain at right angles to the board.
- Remove the tape and cardboard and solder the remaining make header pins to the board.

Solder the remaining components to the shield board. Make sure that the LEDs are inserted in the correct orientation (longer lead to the + side of the component).

Insert the second set of stacking header pins into the female headers of the Uno board. Then *CAREFULLY* insert the shield board header pins into the stacking headers, making sure that all pins are correctly inserted into their respective sockets.

Plug a USB cable into the Uno's USB-B connector and connect the other side of the USB cable to a computer or USB power supply. The Green LED should light up, indicating that the shield board has power.

You can test that you have soldered the Red LEDs in the right way using a male-male jumper wire:

- Insert one end of the jumper wire into the +5 volt pin on the Arduino stacking header on the shield board.
- Touch the other end of this jumper wire to pin 6 of the 28 pin socket. The Red LED near this socket should light up.
- Touch the other end of this jumper wire to pin 3 of the 8 pin socket. The Red LED near this socket should light up.

The hardware assembly of the BBP is now complete.

CONFIGURING THE ARDUINO IDE FOR PROGRAMMING "BARE BONES" CHIPS.

It is necessary to add support for "bare bones" Atmega329(P) and ATtiny85 chips to the Arduino IDE in order to use the IDE to compile and flash code to these chips.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

Adding support for the ATmega328(P) “Bare Bones” chip.

You first need to add a “hardware/breadboard” folder to your Arduino IDE Sketchbook. You can find the location of your Arduino sketchbook using the Arduino IDE. Open the Arduino IDE and select “File : Preferences”. You will see the location of your sketchbook here (figure 4):

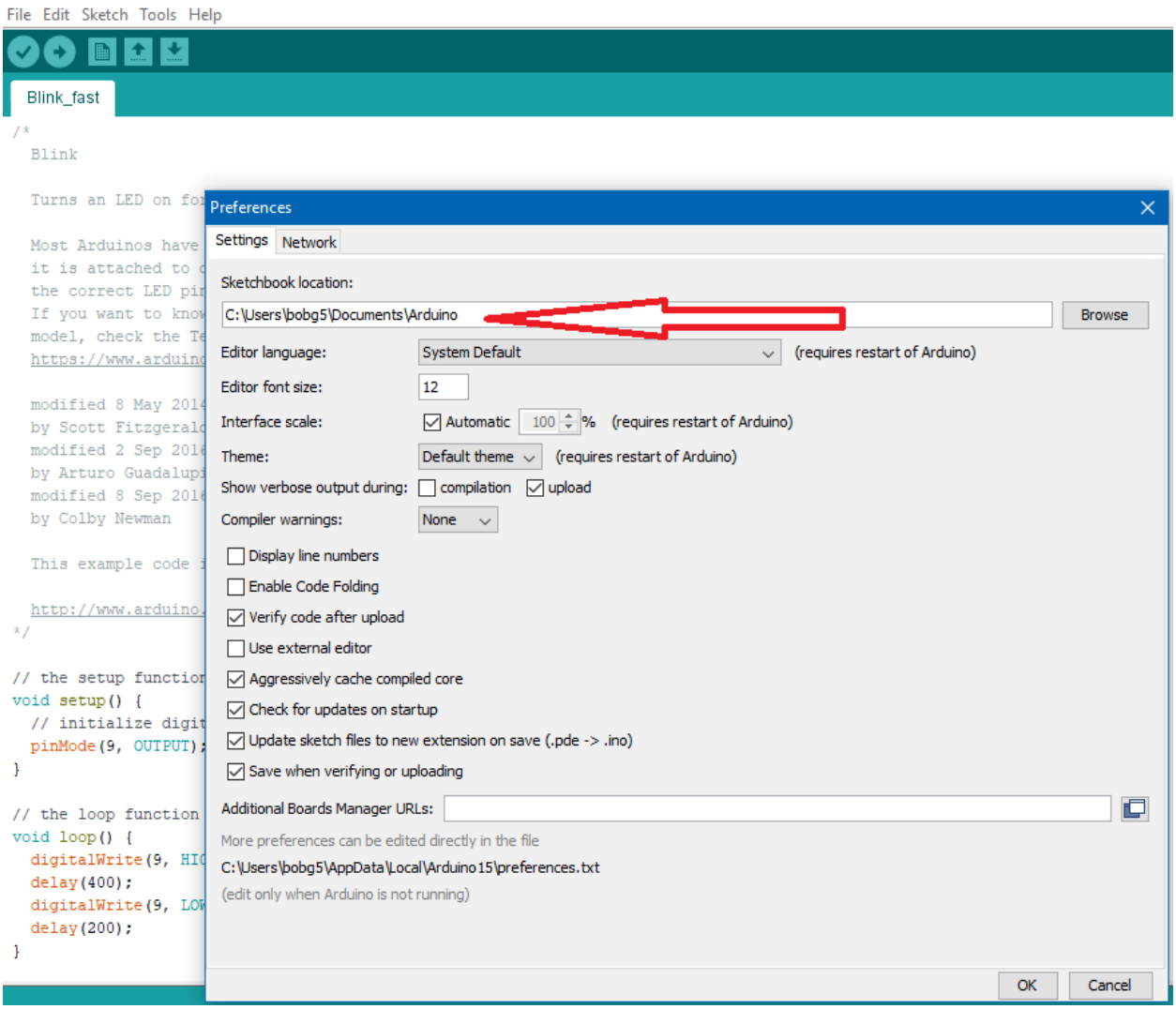


Figure 4. Finding the Sketchbook Location.

Open the sketchbook location on your computer using your computer’s file explorer. See if there is already a folder called “hardware” here. If not, create an empty folder called “hardware”.

Download the following zip file from the Arduino website:
https://www.arduino.cc/en/uploads/Tutorial/breadboard-1-6-x.zip

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to “Terms of Use, License and Disclosure”*

Open this zip file and extract the contents. The contents will be a folder called “breadboard” with some subfolders and other files inside of it. Drag and drop the “breadboard” folder inside of the “hardware” folder in your Arduino sketchbook.

If you plan to use only the -P chips, then you don’t need to do anything else in this step. If you plan to use non-P chips, follow these additional steps (figure 5):

- Open the “breadboard” folder that you just installed into the “hardware” folder in your Arduino sketchbook location.
- Open the “avr” folder.
- Open the file “boards.txt” in any text editor (e.g. Notepad on a Windows PC). It will look like the following figure:

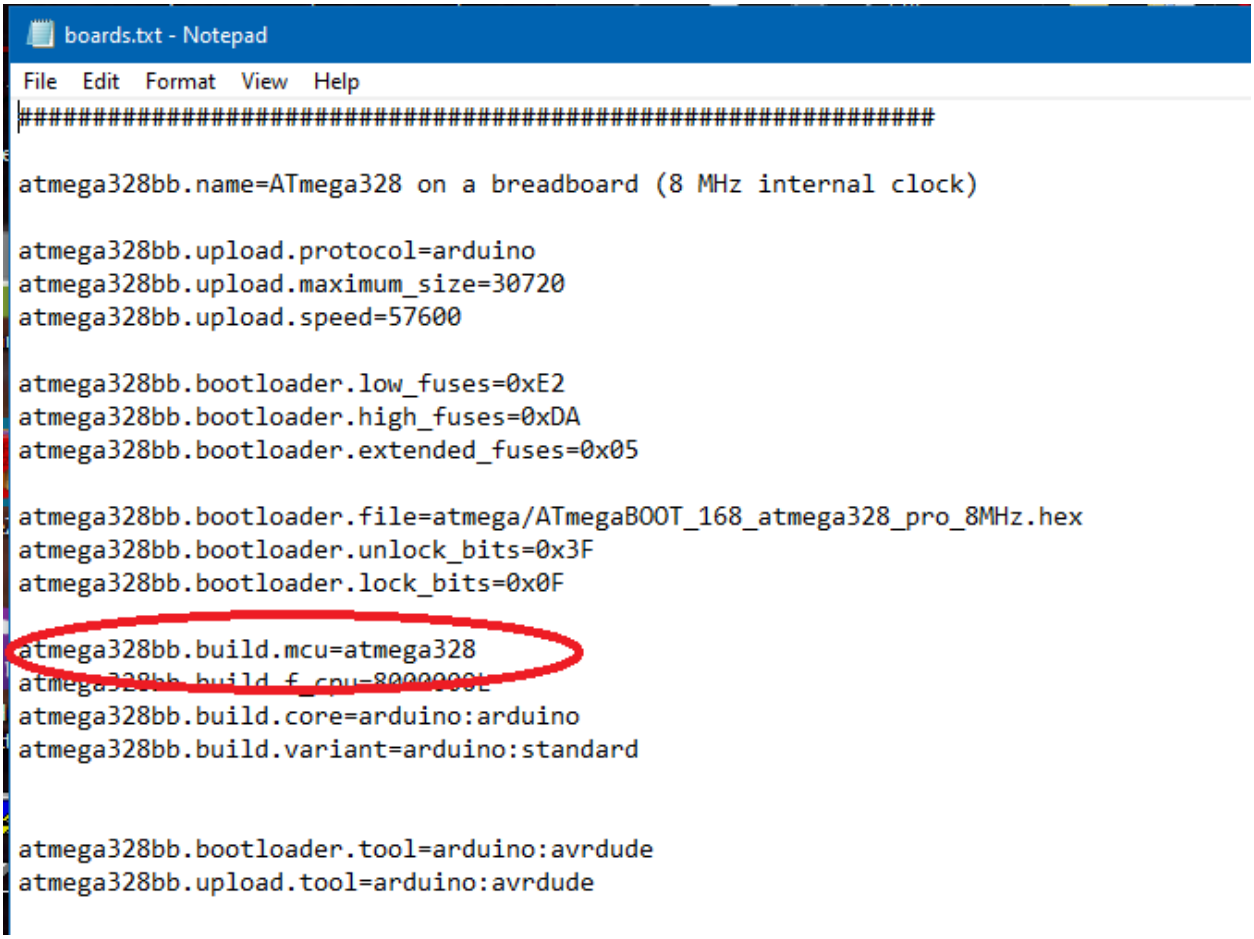


Figure 5. ATmega328 Config File.

Note the circled line in figure 5. The downloaded file will say “atmega328bb.build.mcu=atmega328p”. Note the “p” at the end of this line. This tells the Arduino IDE that the bare bones chip will be an ATmega328P chip. Remove the “p” at the end of this line (as shown in the figure) if you want to upload code to non-P chips. SAVE the edited file and exit your text editor.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to “Terms of Use, License and Disclosure”*

Adding support for the ATtiny85 “Bare Bones” chip.

Use the following steps to add support for “bare bones” ATtiny 25/45/85 chips to the Arduino IDE.

1. Open the Arduino IDE and go to **File > Preferences**.
2. In the **Additional Boards Manager URLs** field, paste the following URL:

   https://raw.githubusercontent.com/damellis/attiny/ide-1.6.x-boards-manager/package_damellis_attiny_index.json
   If you have other URLs, separate them with a comma.

3. Click **OK**.
4. Go to **Tools > Board > Boards Manager**.
5. Search for "attiny" and install the board package ("attiny" by David A. Mellis).

Reset the Arduino IDE.

In order to activate the newly installed “bare bones” chip support in the Arduino IDE, you need to close and then re-open the IDE. After re-opening the Arduino IDE, Select “Tools”, then “Board” and see a choice for “breadboard-avr (in Sketchbook)”. You should have a selection of “Atmega328 on a breadboard (8 MHz internal clock)”; see figure 6, below. This confirms support for the Atmega328(P).

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to “Terms of Use, License and Disclosure”*

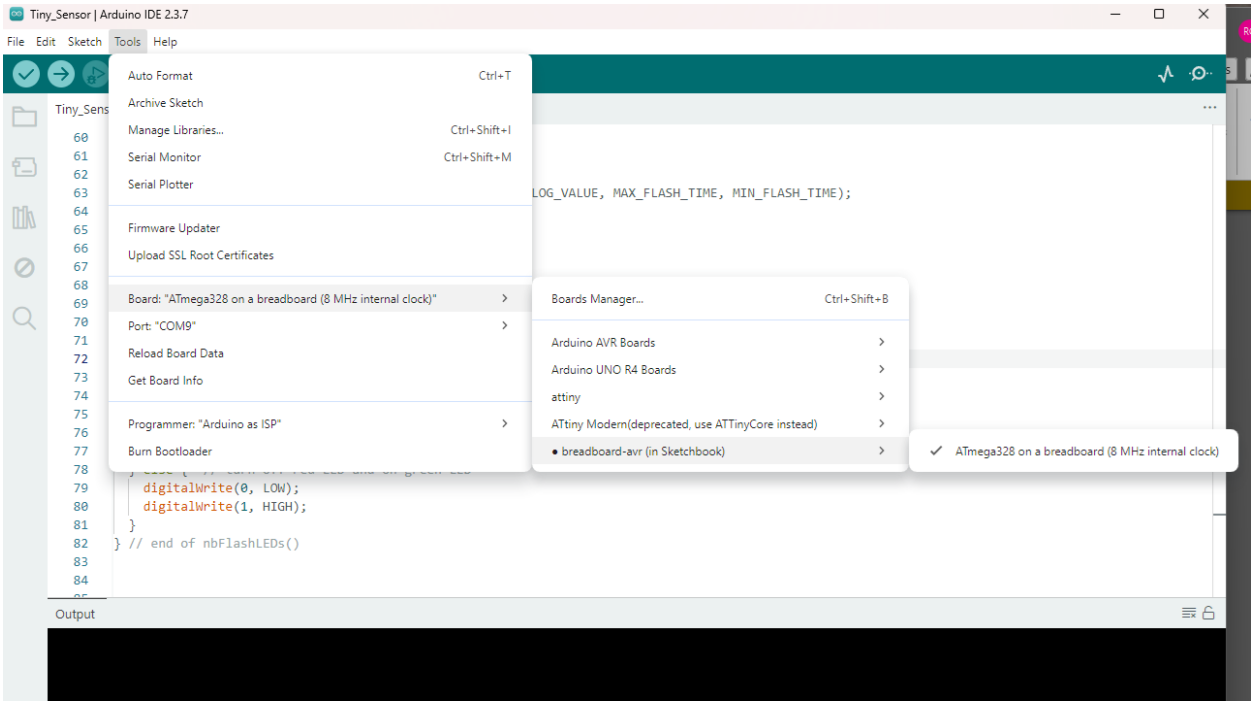


Figure 6. Selection for ATMega328 in Arduino IDE.

Next, again select “Tools”, then “Board” and see a choice for “attiny”. You should have a selection of “ATtiny25/45/85”; see figure 7, below. This confirms support for the ATtiny85.

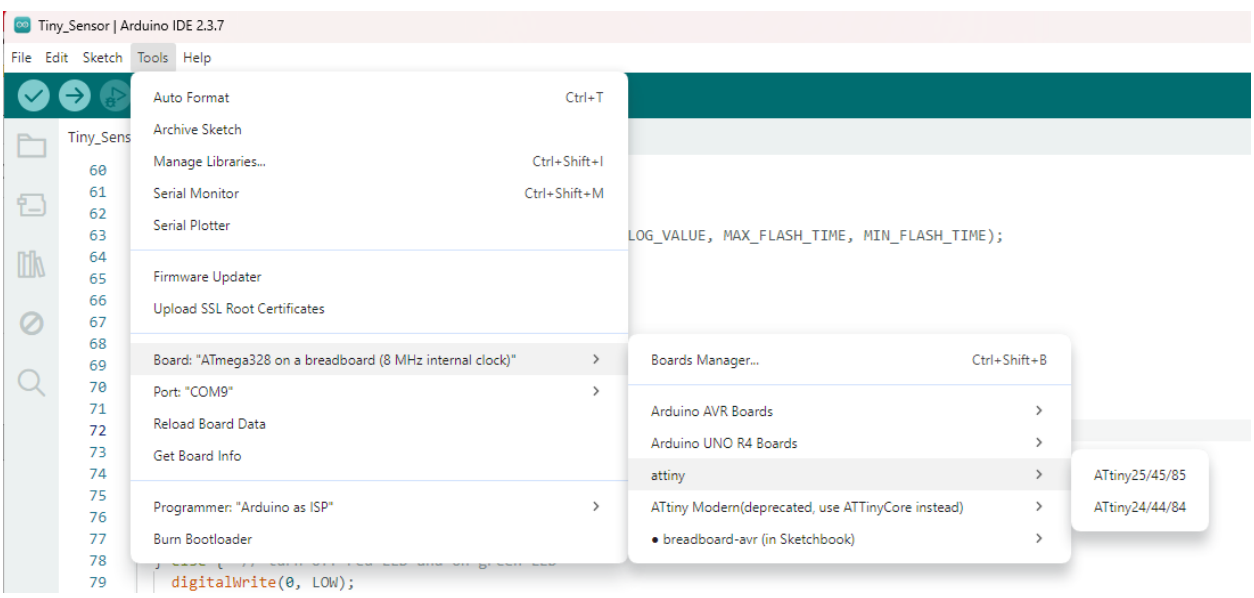


Figure 7. Selection for ATtiny85 in Arduino IDE,

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to “Terms of Use, License and Disclosure”*

FLASHING THE ARDUINOISP PROGRAM TO THE UNO.

Open your Arduino IDE.

As an *optional* first step, we recommend that you enable “verbose” status on the IDE when uploading code. Open the Arduino IDE preferences and check the “upload” check box, as shown in figure 8, below. Then click “OK” to exit the preferences.

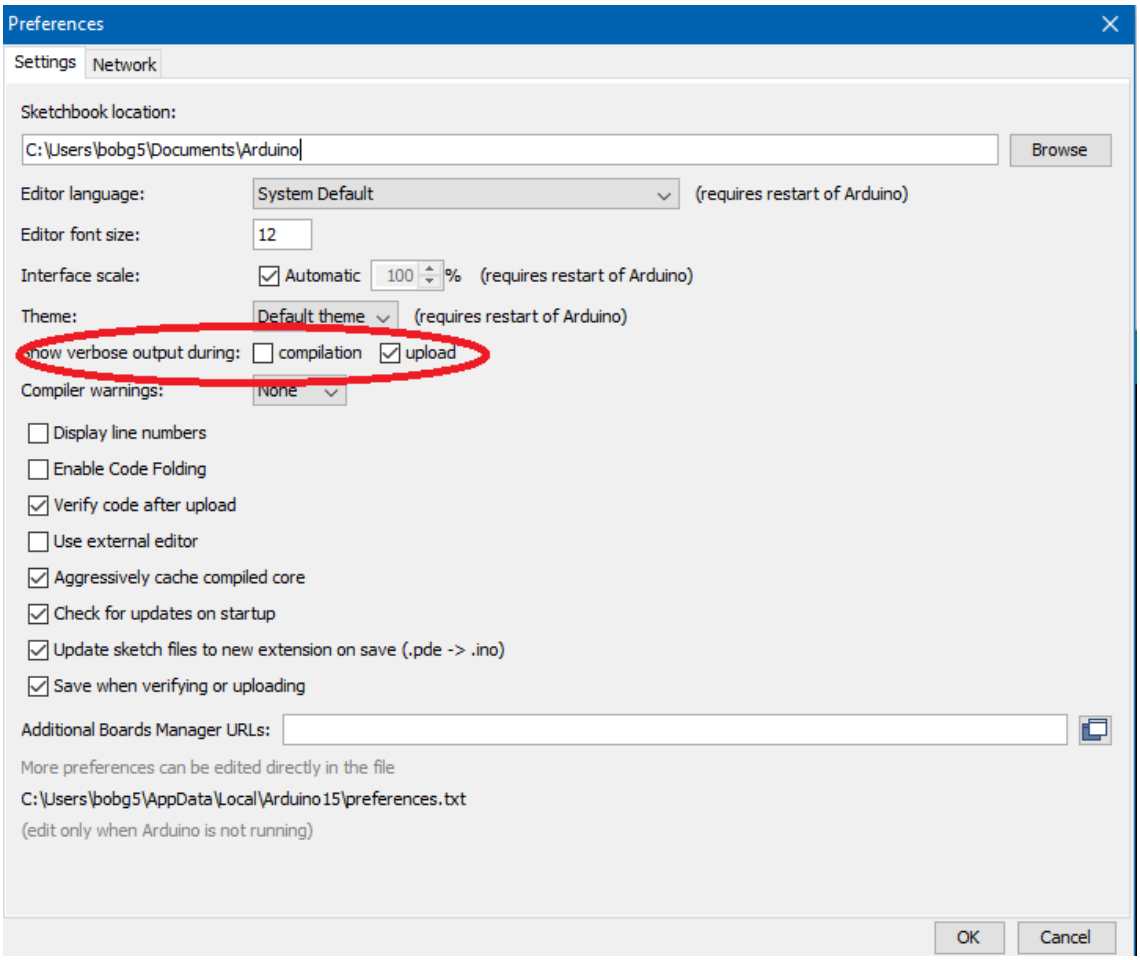


Figure 8. Select “Verbose” In the Arduino IDE.

Next, you need to load the file “ArduinoISP” into the IDE, see figure 9, below. Select “File: Examples:11. ArduinoISP: ArduinoISP”. The IDE code window will show the source code for the Example file called “ArduinoISP”.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to “Terms of Use, License and Disclosure”*

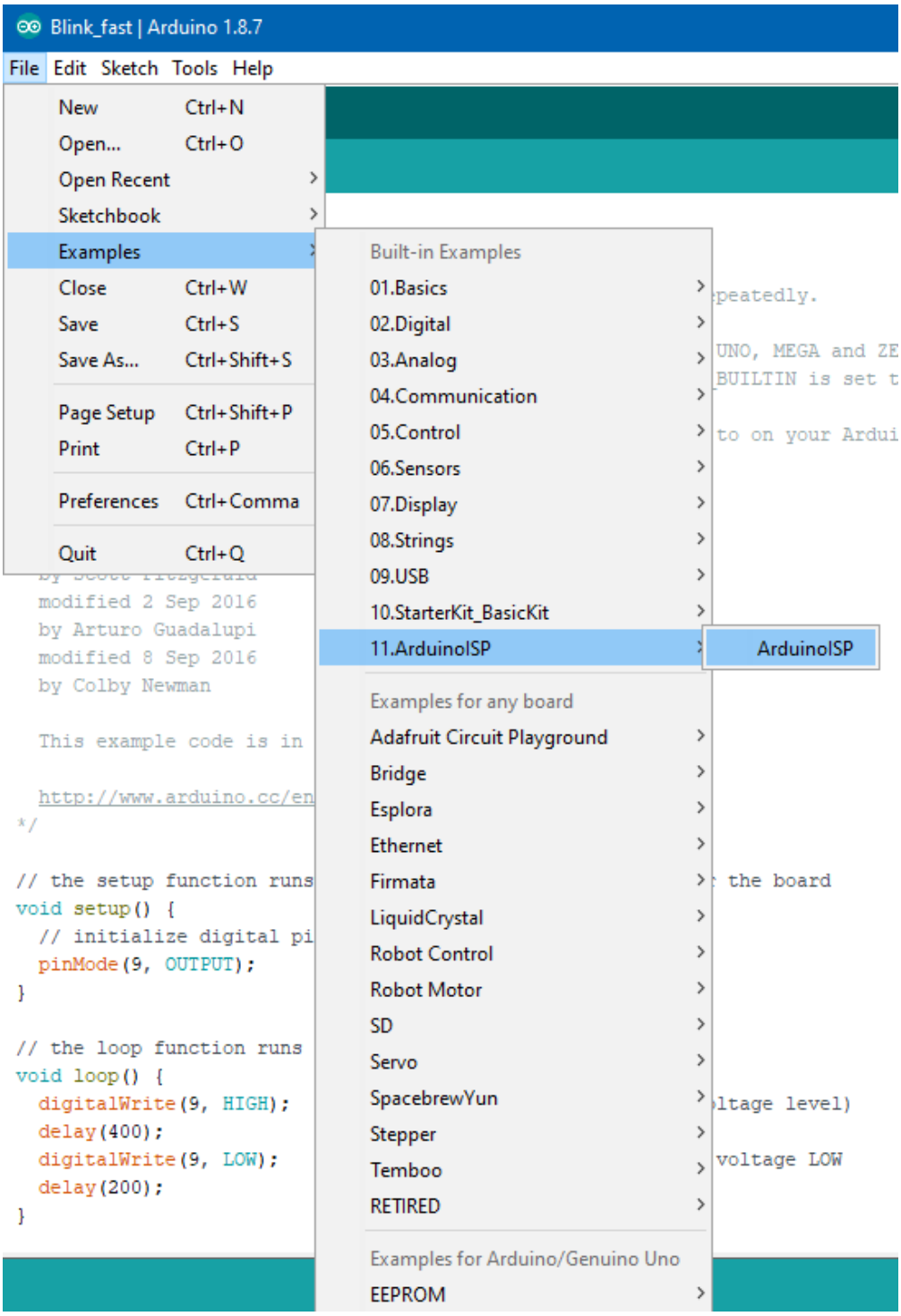


Figure 9. Load the Example Sketch "ArduinoISP" in the Arduino IDE.

Before compiling and uploading "ArduinoISP" to your Uno board, make sure that your Tools settings are correct - see figure 10, below:

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

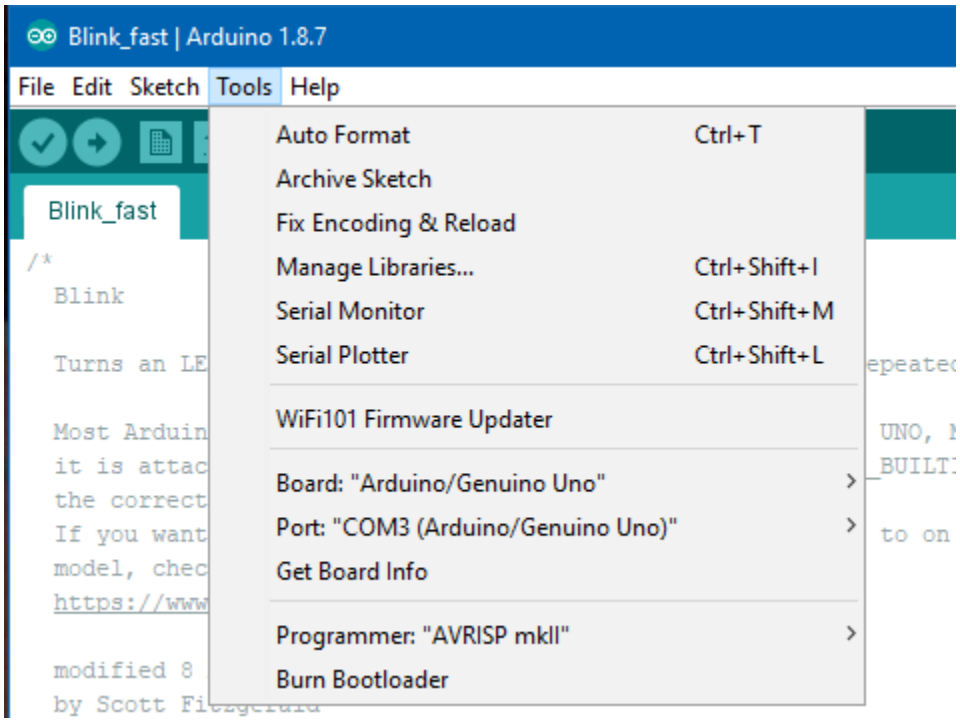


Figure 10. Checking the "Tools" for Flashing ArduinoISP to the Uno.

Note the following in this figure:

- The "Board:" must be "Arduino/Genuino Uno"
- The "Port" will be the COM or tty port that your Uno is connected to (may not be COM 3, as in the above figure).
- The "Programmer" must be set to "AVRISP mkII".

After double checking these Tools settings, click on the Compile and Upload button: the Right facing arrow at the top left of the IDE window. This will flash the ArduinoISP code to your Uno. *<u>NOTE</u>: You only need to do this once - thereafter, the ArduinoISP code will be running when you power up your Uno (unless you change it, of course).*

FLASHING THE ATMega328 or ATTiny

Do we need to add some notes here about using the Sketch Upload Using ArduinoISP ?

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*